The Cyber Threat Reconstruction Engine: Offline Correlation of NTFS Journaling and Network Metadata for Timestomping and Exfiltration Detection

Abstract

We present the Cyber Threat Reconstruction Engine (CTRE), an offline tool for digital forensics and incident response that reads three kinds of pre-collected evidence, correlates them, and produces one chronological timeline. The three inputs are an NTFS master file table ($MFT), an NTFS change journal ($UsnJrnl:$J), and a classic network capture (.pcap). These artifacts are usually examined with separate tools, which makes it easy to miss a link between a file that changed on disk and traffic that left the machine at the same moment. CTRE reads each input read-only, parses it, and then checks for two things: timestamps that have been altered to hide when a file was created or modified, and large outbound transfers that line up in time with a disk change. The system is built from the Python standard library alone, runs either as a local web page or from the command line, and ships with a generator that writes synthetic artifacts so the detectors can be exercised without real evidence. We deliberately exclude any extraction of TLS session keys from memory and any decryption of captured traffic; CTRE uses only the metadata already visible in a capture. On a planted test case the engine identified a timestomped file through three independent rules and tied an outbound transfer to the matching disk change, and a cross-check against an independent packet library confirmed the metadata it reports.

Index Terms

Digital forensics, incident response, NTFS, $MFT, $UsnJrnl, change journal, timestomping detection, anti-forensics, PCAP metadata, timeline analysis, data exfiltration, DFIR.

I. Introduction

A. Background and Motivation

A forensic investigation of a Windows host usually draws on several kinds of evidence at once: the file system, volatile memory, and network traffic. In practice each kind tends to be examined with its own tool, and the results are reconciled by hand. This separation makes some questions hard to answer, because the answer lives in the relationship between two artifacts rather than in either one alone. A file that appears on disk and a burst of outbound traffic a few seconds later are only suspicious together; viewed apart, each can look ordinary.

Two problems motivated this work. The first is that an attacker who wants to hide activity will often alter file timestamps, a technique known as timestomping, so that a malicious file blends in with older system files. The second is that exfiltration over an encrypted channel hides its payload but not its shape: the time, size, and destination of a transfer are still visible in a capture. CTRE was built to use both of these facts, by comparing timestamps against a tamper-resistant record and by lining up network activity with disk activity on a single timeline.

B. Problem Statement

The work addresses a specific question: given copies of a host's $MFT, its change journal, and a packet capture, can a tool read all three safely, detect timestamp manipulation that a single-artifact view would miss, and surface disk and network events that are related in time, without decrypting any traffic and without altering the evidence?

C. Project Objectives

The implementation has six objectives, listed in Table I. They cover safe handling of evidence, the two detection goals, a reproducible way to test the detectors, and output that an analyst can read and share.

TABLE I: Project Objectives and Realization

ID	Objective	Implementation
O1	Safe evidence handling	Read-only access, standard library only, no network use
O2	Timestamp tamper detection	$MFT compared against $FILE_NAME and the change journal
O3	Exfiltration correlation	Outbound flows matched to nearby disk changes
O4	Keyless network analysis	Flow, DNS, and TLS server-name metadata only
O5	Reproducible testing	Generator writing real on-disk structures; 20 tests
O6	Readable output	Static HTML report, JSON export, local web UI

D. Scope and Non-Scope

In scope are: parsing the three artifact types; four timestomping rules with a severity and a written reason for each; correlation of large outbound transfers with disk changes; a unified timeline; and functional tests over synthetic data. Out of scope, by design, is the reading of TLS session secrets from a memory dump and the decryption of captured traffic. The same code that decrypts a lab capture with the analyst's own keys would decrypt anyone's traffic given a key store and a capture, so that building block is left out regardless of how it is packaged. The rest of the workflow does not depend on it. Also out of scope are formal proofs, live acquisition, and support for capture or file-system formats other than those named here.

E. Contributions

The codebase contributes a dependency-free, read-only correlation pipeline for three common Windows artifacts; a set of timestomping rules that cross-check the easily altered $MFT timestamps against the kernel-maintained change journal, with guards against the two most common false positives (a journal that has wrapped, and an $MFT record reused after deletion); a keyless network stage that reads only observable metadata; and a synthetic artifact generator that writes the same on-disk structures the parsers read, so the generator and parsers validate each other.

II. System Context

A. Design Intent

CTRE treats every input as untrusted evidence. Files are opened read-only and parsed defensively, so that a malformed or hostile artifact is skipped rather than allowed to crash the run or reach outside the parser. The tool needs nothing beyond a Python interpreter, which keeps it easy to audit and easy to run on an isolated analysis machine.

B. Component Overview

Table II lists the main components and what each one does. The pipeline is the same whether the tool is driven from the command line or from the local web page; both call one shared analysis function.

III. System Architecture and Methodology

Figure 1 shows the overall pipeline. The three artifacts are read by independent parsers; the parsed output then feeds two analysis stages, whose results are merged into one timeline and a report.

TABLE II: Core Components

Component	Role
$MFT parser	Reads FILE records, applies fixups, extracts timestamps
$UsnJrnl parser	Reads USN_RECORD_V2 change-journal entries
Timestomp detector	Compares timestamp sources and the journal history
PCAP metadata reader	Builds flow records, reads DNS names and TLS SNI
Correlator	Builds the timeline and matches exfil to disk changes
Report generator	Writes a static, escaped HTML report and JSON
Web server	Loopback-only local UI over the shared pipeline
Artifact generator	Writes synthetic $MFT, $UsnJrnl, and pcap test data

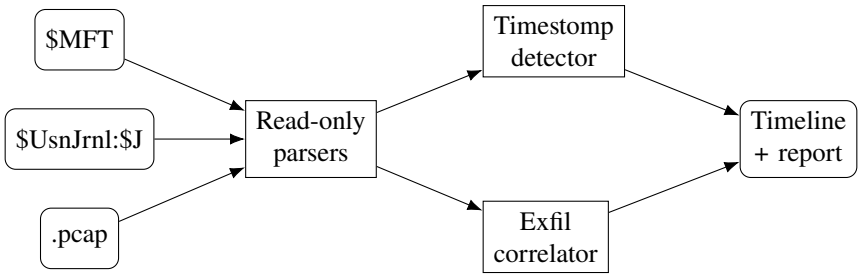


Fig. 1: CTRE pipeline: three artifacts are parsed read-only, analyzed for timestamp tampering and for disk/network correlation, and merged into one timeline and report.

A. Disk Parsing

The $MFT parser reads each FILE record, applies the update-sequence (fixup) array that NTFS uses to detect torn writes, and then walks the record's resident attributes. It extracts the timestamps from two attributes: $STANDARD_INFORMATION, which holds the times shown to users and which anti-forensic tools can change easily, and $FILE_NAME, which the kernel maintains and which is harder to alter from user space. A record that does not parse is skipped, because evidence is untrusted input.

The $UsnJrnl parser reads the change journal, which is an append-only log that the kernel keeps of file-system changes. Each record names a file, a time, and a set of reason flags such as create, rename, data-overwrite, and delete. An extracted journal is often several gigabytes, most of it sparse zero padding, so the parser memory-maps the file and jumps over the zero regions instead of reading the whole thing into memory.

B. Timestomping Detection

The detector compares the two timestamp sources against each other and against the journal. It applies four rules, shown in Table III. Each result carries a severity and a plain-language reason, so the output can go straight into a report rather than being a bare flag.

Figure 2 shows the checks applied to each file. The checks run in order, and a file can raise more than one finding.

Two of these rules need care to avoid false positives. The change journal is a ring buffer, so on a real volume the oldest surviving record is usually not the file's creation. The SI_BEFORE_JRNL rule therefore fires only when the journal holds an explicit file-creation record for the file, not when it simply holds an old edit. NTFS also reuses an $MFT record number after a file is deleted, so journal records from a previous file can share a record number with the current one. The detector matches on the record's sequence number as well, so that history belonging to a deleted file is not read as tampering on the file that replaced it.

TABLE III: Timestomping Rules

Rule	What it flags
SI_BEFORE_FN	$STANDARD_INFORMATION created earlier than $FILE_NAME created
SI_BEFORE_JRNL	SI creation time earlier than the file's first journal record
ZERO_SUBSECOND	Several SI timestamps with their sub-second precision zeroed
NO_JRNL_FOR_CHANGE	SI modified time newer than any journal entry for the file

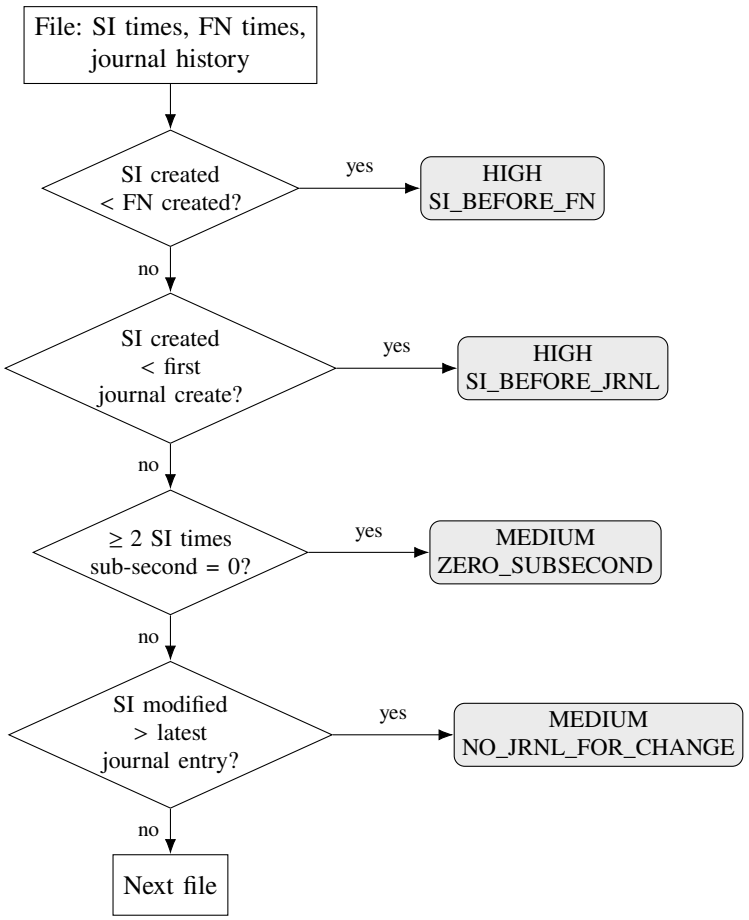


Fig. 2: Timestomping checks applied per file. Each rule compares the easily altered $STANDARD_INFORMATION (SI) times against the $FILE_NAME (FN) times and the change journal.

C. Network Metadata and Correlation

The pcap reader parses the classic libpcap format with Ethernet and IPv4 frames. It builds one flow record per connection, holding the endpoints, ports, packet and byte counts per direction, and the first and last time the flow was seen. It also reads two pieces of cleartext that are present without any keys: DNS query names, and the server name sent in a TLS ClientHello. It performs no decryption.

The correlator looks for a large outbound transfer whose start time falls within a configurable window of a disk change recorded in the journal, which is the shape of a stage-and-exfiltrate step. All of the disk and network events are then merged into one timeline sorted by time, so that a file creation, its journal record, and a later outbound transfer to an external host appear in sequence. Figure 3 shows the matching logic.

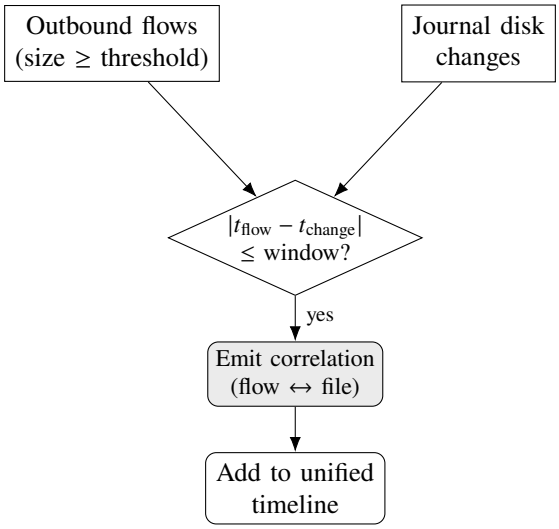


Fig. 3: Exfiltration correlation: each large outbound flow is paired with any disk change that falls within the time window, and the pair is added to the timeline.

IV. Implementation Details

A. Parsers and Output

Every parser is written against the structure it reads and is tolerant of malformed input. Timestamps are converted from the Windows FILETIME format (100-nanosecond intervals since 1601) into timezone-aware values, so that disk and network events share one time base. The report is a single HTML file with no external assets. Every value drawn from the evidence is HTML-escaped before it is written, so a hostile file name cannot inject markup into the report. The same results are also available as JSON.

B. Local Web Interface

Running the tool with no arguments starts a server on the loopback address and opens it in a browser. The analyst drags in the artifacts or loads a built-in sample case, sets the options, and runs the analysis; the findings, correlations, and timeline appear in the page, and the report or the JSON can be downloaded.

Because any web page open in a browser can try to reach a local port, the server is written defensively. It binds to the loopback address only. It checks that the `Host` header names the loopback address, which blocks DNS-rebinding access. It requires a custom request header on every API call; a browser will not attach a custom header to a cross-site request without a preflight, and the server answers no preflights, so other sites cannot drive it. It sends a strict Content-Security-Policy with every page, and it deletes uploaded artifacts when it stops. In the browser, every value from the evidence is inserted through `textContent` rather than as HTML.

V. Methodology and Verification

A. Test Strategy

The project ships a generator that writes synthetic artifacts in the real on-disk structures the parsers expect: FILE records with fixups and the two timestamp attributes, USN_RECORD_V2 journal entries, and a libpcap capture with DNS, a TLS ClientHello, and a sized outbound transfer. One file in the synthetic $MFT is deliberately timestomped, and the capture holds a transfer timed to match a disk change, so the detectors and the correlator have real cases to find. Because the generator writes the same structures the parsers read, a round-trip test confirms that each parser recovers exactly what was written.

B. Cross-Check

The pcap reader was checked against scapy, a separate and widely used packet library. Packet count, DNS query name, TLS server name, and the per-direction byte totals all matched. This gives some confidence that the reader's output reflects the capture rather than an artifact of the parser.

C. Reproducibility

The verified result is reproduced as follows: generate the sample artifacts, run the analysis to produce the report, and run the test suite. The command-line form takes the three artifact paths and a report path; the web form loads the same sample case from a button. Each step is observable, so a failure can be traced to one stage rather than to the pipeline as a whole.

D. Assumptions and Threats to Validity

The results were obtained on generated data, not on an image or capture from a real machine. Byte counts in the flow records include frame headers, which is the quantity the scapy cross-check measured against. The pcap reader handles the classic libpcap format with Ethernet and IPv4 only; it does not read pcapng, IPv6, or other link types, and it reports a clear error and a conversion hint rather than failing silently. The timestomping rules are heuristics: they encode patterns that strongly suggest tampering, but a zeroed sub-second field, for example, can also come from a file copied off FAT media or extracted from a ZIP archive, so a finding is evidence to examine rather than proof.

VI. Results

On the planted sample case the engine parsed 2 $MFT entries, 4 change-journal records, and 122 packets forming 2 flows. It reported three timestomping findings, all on the planted file `svchost.exe`: two at high severity (SI_BEFORE_FN and SI_BEFORE_JRNL) and one at medium severity (ZERO_SUBSECOND). It reported two correlations tying an outbound transfer of roughly 160 KB to the same host to the two journal records for that file. The normal file in the sample was not flagged. Table IV summarizes the run.

TABLE IV: Sample-Case Analysis Summary

Measure	**Value**
$MFT entries parsed	2
Change-journal records parsed	4
Packets / flows parsed	122 / 2
Timestomping findings (high / medium)	3 (2 / 1)
Exfiltration correlations	2
Normal file flagged	no

The timeline from the same run tells the story in order: `svchost.exe` shows an $MFT creation time in 2019, while the change journal records its real creation in 2026, followed a little later by a DNS query for the external host, a TLS ClientHello to it, and the outbound transfer. The 2019 time is the planted tamper; the journal is what exposes it.

The test suite holds 20 tests and all pass. They cover the FILETIME conversion, a round trip through each parser, the four detection rules, the correlation, report escaping, and a full upload-analyze-download cycle against the running web server over HTTP. Two of the detection tests come from a validation pass and confirm the false-positive guards: a journal that has wrapped, and an $MFT record reused after a deletion, no longer produce a timestomping finding. As a rough measure of the journal parser's behavior on large input, a one-gigabyte sparse journal prefix parsed in about 4.5 seconds, because the parser skips the zero regions rather than reading them.

VII. Discussion

The sample run shows the value of the cross-artifact view. Each timestomping rule on its own could be argued away, but the $FILE_NAME time and the journal's creation record agree with each other and disagree with the tampered $STANDARD_INFORMATION time, which is hard to explain as anything but a backdated timestamp. The correlation adds a second thread: the same file that was backdated also lines up in time with an outbound transfer to an external host. Neither the disk evidence nor the network evidence makes that case alone.

The decision to exclude traffic decryption shapes what the tool can and cannot say. CTRE cannot report the contents of an encrypted transfer, only its time, size, and destination, plus the server name offered in the clear. For the exfiltration question that is often enough, because the suspicious signal is a large transfer to an unexpected host at a telling moment, not the bytes themselves. The limit is deliberate and keeps the tool to evidence that any analyst can read from the capture without keys.

The results are a functional demonstration on controlled data, not a field evaluation. They show that the pipeline works end to end and that the detectors fire on a known case and stay quiet on a benign one. They do not establish detection rates on real intrusions, and the heuristics would need tuning against real volumes before the findings could be read as more than leads.

VIII. Ethical and Safety Considerations

CTRE operates on copies of already-collected artifacts and never writes to them. It makes no network connections of its own, and the local web server listens only on the loopback address with the protections described above. The excluded decryption stage is the main safety decision in the project: a tool that reads session keys from memory and decrypts captured traffic is general-purpose interception, and it was left out for that reason. An analyst who legitimately holds the keys for a lab capture can still decrypt it with existing, purpose-built tooling; CTRE does not need to be that tool to do its job.

IX. Limitations and Future Work

The current limitations are the ones named under threats to validity: generated-data testing only, classic libpcap with Ethernet and IPv4 only, and heuristic rules rather than proofs. Useful extensions include support for pcapng and IPv6, more timestomping rules (for example comparing $STANDARD_INFORMATION against $FILE_NAME modification and access times, not only creation), handling of several volumes or hosts at once, and evaluation against captured real-world cases to measure how often the rules fire correctly.

X. Conclusion

CTRE reads three common Windows artifacts offline and read-only, detects timestamp manipulation by checking the easily altered $MFT times against the kernel's change journal, and ties large outbound transfers to disk changes on a single timeline, all without decrypting any traffic. On a planted case it found the tampered file through three independent rules, left the benign file alone, and connected an outbound transfer to the matching disk activity, and its network output agreed with an independent packet library. The result is a small, auditable, dependency-free tool that demonstrates the cross-artifact approach. The next step is to move from controlled data to captured real cases, so that the detection rules can be measured rather than only demonstrated.

References

[1] B. Carrier, *File System Forensic Analysis*. Addison-Wesley, 2005.
[2] H. Carvey, *Windows Forensic Analysis Toolkit*, 4th ed. Syngress, 2014.
[3] Microsoft, "Change Journals and the USN_RECORD_V2 structure," Windows Dev Center documentation.
[4] Microsoft, "FILETIME structure," Windows Dev Center documentation.
[5] B. Schneier and others, discussion of anti-forensic timestamp manipulation; see also the MACB time model in [1].
[6] The Tcpdump Group, "PCAP Capture File Format" (libpcap), tcpdump.org.
[7] D. Eastlake, "Transport Layer Security (TLS) Extensions: Extension Definitions," IETF RFC 6066, Jan. 2011 (Server Name Indication).
[8] P. Biondi, "Scapy: Packet crafting and manipulation for Python," scapy.net.
[9] Python Software Foundation, "The Python Standard Library" (struct, mmap, http.server, importlib.resources), docs.python.org.